Tools & Miscellaneous

Linux – *dmesg (I)*

- **possibility to analyse the kernel-level details**
  - analysis of the boot process
    - display messages from the kernel's ring buffer
    - system boot, hardware detection, driver initialization, and kernel errors
  - history of kernel interactions
    - kernel events like hardware connections, memory allocations, and peripheral issues.
  - system debugging (related to hardware)
    - USB devices, disk errors, CPU issues
    - hardware failures or driver errors are present in kernel logs
  - hardware configuration
    - detailed HW information, for configuration and/or troubleshooting

© A.Atzeni (Politecnico di Torino, 2024)

Linux – *dmesg (II)*

- filtering
  - cooperation with command-line tools
  - e.g. *`Grep and dmesg`* to focus on USB, CPU, memory-related logs, …
    - `dmesg | grep usb`
    - for USB device connections, helping to debug issues with USB peripherals.
- timeline reconstruction
  - `-T` option for human-readable timestamps for kernel messages

© A.Atzeni (Politecnico di Torino, 2024)

Linux – *kill*

- **It is 1) a function for signal delivery 2) a shell command to**
  - send signals to processes to instruct them to perform specific actions (e.g., terminate, pause, or continue execution).
  - targets specific processes (or process groups) using the process identifier (PID)
- **signals example**
  - SIGKILL (forceful termination), SIGTERM (graceful termination), SIGSTOP (pause), and SIGCONT (resume)
- **some signals (e.g., SIGTERM) can be caught and handled by the process, allowing it to perform cleanup tasks**
  - user must have the necessary permissions to send a signal to a process
    - In general, sending signals to processes owned by other users may not be allowed
- **essential for controlling processes in Linux environments**

© A.Atzeni (Politecnico di Torino, 2024)

Linux – *kill alteration*

- **incorrect process responses**
  - improperly handled signals drive to incorrect behavior (e.g. incomplete shutdowns)
- **privilege escalation**
  - If modified to bypass permission checks, allow unauthorized signals sending to processes owned by other users
    - critical services manipulation, breach of security policies
- **persistence**
  - ignored/mishandled signals lead to runaway process (system degradation) and persiostent process (e.g. rootkits can instruct SIGKILL to be ignored)
- **monitoring compromission**
  - signals are used to trigger logging or monitoring actions (e.g., a service may reload its configuration upon receiving SIGHUP)

© A.Atzeni (Politecnico di Torino, 2024)

Linux – *kill alteration*

- **incorrect process responses**
  - improperly handled signals drive to incorrect behavior (e.g. incomplete shutdowns)
- **privilege escalation**
  - If modified to bypass permission checks, allow unauthorized signals sending to processes owned by other users
    - critical services manipulation, breach of security policies
- **persistence**
  - ignored/mishandled signals lead to runaway process (system degradation) and persiostent process (e.g. rootkits can instruct SIGKILL to be ignored)
- **monitoring compromission**
  - signals are used to trigger logging or monitoring actions (e.g., a service may reload its configuration upon receiving SIGHUP)

© A.Atzeni (Politecnico di Torino, 2024)

Computer Forensics Tools Reqs

CF tool requirements

- **usability**
  - present data at a layer of abstraction useful to an investigator
- **comprehensiveness**
  - present all data to investigator so that both inculpatory and exculpatory evidence can be identified
- **accuracy**
  - tool output must be verifiable with a (small) margin of error
- **deterministic**
  - produce the same output given the same rules and input data
- **verifiable**
  - ability to verify the output by accessing the layer inputs and outputs (verification can be done by hand or via a second tool)

© A.Atzeni (Politecnico di Torino, 2024)

Further CF challenges

- **the forensics uncertainty**
  - anything you do to a system disturbs it
    - work on copies
- **trust no 1**
  - you can never trust the system software
    - work with your tools
- **the history is written by the winners**
  - we can never know the past (or even the present)
    - do not use regular system infos (only)
- **everything is impermanent**
  - soon or late, all the evidences will disappear
    - collect most-volatile first

© A.Atzeni (Politecnico di Torino, 2024)

CF challenges

- **size of storage devices**
- **embedded flash devices**
- **proliferation of operating systems and file formats**
- **multi-device analysis**
- **pervasive encryption (local / network)**
- **cloud computing**
- **RAM-only malware**
- **legal challenges decreasing the scope of forensic investigations**
  - ***S.L. Garfinkel, "Digital forensics research: The next 10 years", Digital Investigation, vol.1, no.7, pp.64-73, 2010, DOI 10.1016/j.diin.2010.05.009***
  - *Plus modern add like to add AI generated data and deepfakes, evolving anti-forensic techniques*

© A.Atzeni (Politecnico di Torino, 2024)

Digital Forensics lab set-up

© A.Atzeni (Politecnico di Torino, 2024)

Location and physical security (I)

- **secure facility**
  - geographical location
    - remote or easily securable
    - free from vulnerabilities to natural disasters or interference
  - implement defense in-depth multi-layered security
    - biometric access (fingerprints, iris scans)
    - keycard entry systems
    - video surveillance (CCTV) and 24/7 security personnel
    - alarm systems integrated with local law enforcement

Location and physical security (II)

- **air-gapped environment**
  - physically isolated system (including the internet)
  - prevent any external connection to workstations or network sections
- **faraday cage rooms**
  - prevent external wireless signals from interfering or compromising data-processing area
- **environmental control**
  - maintain appropriate temperature and humidity for electronic devices (servers, workstations)
  - fire suppression systems to protect from fire
    - ...with no harm to electronic devices
      - e.g. based on Halon or FM-200

Data Protection (I)

- **segregated networks**
  - network segmentation (analysis, data storage, sensitive communication)
    - through VLAN and/or VPN
  - encrypted communication
    - adopting high-grade encryption standard (AES-256, RSA-4096)
- **least-privileged access control**
  - role or attribute based
    - e.g. *admins* for system management
    - *analysts* for digital forensics analysis
    - *supervisors* for auditing and review

Data Protection (II)

- **disaster data recovery readiness**
  - (verified) back-up systems
  - (verified) uninterruptible power supplies (UPS)
- **logging**
  - enable logging for all critical components
    - servers, workstations, databases
      - operating system logs (e.g. *login attemps, file access, administrative actions)*
    - network devices (e.g. *routers, firewalls, switches*)
      - logs on network activity, configurations changes, access attemps
    - end-user devices
      - app-specific logs, user activity, critical transactions

Data Protection (III)

- **auditing readiness**
  - availability of audit trails
    - user specific
      - e.g. login-logout times, executed commands, accessed files, configuration changes
    - application specific
      - e.g. database audit logs to track queries on sensitive information and administrative actions
  - immutable trails storage
    - WORM (write-once- read many)
    - hashing and/or digital signatures
    - Policy-based access-restriction
      - e.g. based on separation-of-duty principle (different roles to r/w)

Hardware requirements (I)

- **high-performance workstations**
  - fast multi-core processors
  - consistent RAM (128GB or more)
  - high-speed SSDs (many TBs)
- **storage forensic servers**
  - large-scale storage for forensic images (tens of Terabytes organised in RAID HD and backup servers)
    - secure evidence storage with redundancy
    - write-once (WORM) storage for legal integrity
- **HW acceleration for specific tasks**
  - e.g. GPU for password cracking (Password Cracking Hardware), Data Carving
- **Faraday bags**

Hardware requirements (II)

- **network forensics hardware**
  - packet capture appliances
    - capture, store, and analyze network traffic at high speed,
    - often with forensic-grade reliability (full packet capture, indexing, timestamping, ...)
  - dedicated network taps
    - passive hardware devices to capture traffic without altering.
    - stealthy, reliable, and legally defensible (no packet loss or interference when properly deployed)
- **virtualization**
  - isolated machines for sandbox systems
    - e.g. malware analysis
- **power supplies & adapters & cable and manuals**

Hardware requirements (III)

- **write-blockers**
- **imaging devices**
  - e.g. tableau comprehensive imaging
- **mobile device acquisition tools**
  - e.g. Cellebrite UFED

Source: *https://siliconforensics.com/products/forensic-hardware/tableau-instant-imaging-response-kits.html*

Source: *https://en.wikipedia.org/wiki/Cellebrite_UFED#/media/File:Ufed_mobile_phone_imaging_device_(8661348282).jpg*

Hardware requirements - UFED

- **Universal Forensic Extraction Device**
  - complete physical device storage image extraction
  - bit-by-bit copy of flash memory
  - access to hidden files and directory
  - access to specific content (contacts, SMS, call logs, multimedia, ...)
  - (with credential knowledge) can extract cloud data associated with the mobile
    - (e.g. iCloud, Google drive, social media like instagram and Facebook, …)
  - can extract data from many apps (e.g. *WhatsApp, FB messenger, Snapchat)*
  - can support encryption/locking bypass
    - exploiting brute-forcing and mobile vulnerabilities

Software

- **Forensics lab must provide**
  - flexibility and scalability
    - due to different scenarios requirements
  - provide safe environments
    - where evidences can be analysed with no system integrity compromise
  - easy recovery after a "failure scenario"
    - because some tests might be pretty "invasive"…
  - when possible, should have a "roaming soul"...
    - ...to go, when possible, to the crime scene

Portable OS

- **create a trusted environment**
  - not related to the one under analysis
- **many solutions based on linux distros “live CDs” (often Debian or Ubuntu based)**
  - security general-purpose
    - GRML, Kali, ...
  - forensics special-purpose
    - CAINE, Helix, SIFT, DEFT, Tsurugi, …
- **also in a virtual machine**
  - copy the evidences in the VM
  - create virtual disks
    - and attach them to the virtual environment

Mounting (I)

- **the process of making a file system accessible**
  - suitable to interact with
    - local file systems (e.g. ext4, NTFS, HFS, …)
    - external physical devices (like usb drives, DVDs, SDs, ...)
    - network file systems (like NFS, SMB, …)
- **at a specific "point" in the directory tree**
  - This is the root of the "new" file system
  - i.e. you can navigate it from that "point"
  - known as "*mount point*"
    - e.g. */mnt/usb*

Mounting (II)

- **mounting process involves**
  - checking of the file system on the storage medium for errors
  - loading the file system's metadata
    - how files and directories are organized
  - associating the "new" file system with a mount point in the existing directory hierarchy
- **once mounted, the operating system can read from and write to the mounted file system with usual operations**
- **linux example (shell commands):**
  - *mount /dev/sdb1 /mnt/usb*
  - *ls –la /mnt/usb*

Mounting (III)

- **unmounting is the reverse operation**
  - detaches the file system from the directory structure
  - crucial for safely removing storage device due to caching and buffering techniques for performance improvement
    - ongoing I/O operation
    - write-behind cache
      - delayed writing changes to disk
    - open files in active processes
      - internal file system structures (e.g. file pointers) may become inconsistent if the file system is brutally removed
    - Inconsistent integrity information
      - metadata that track the state of the file system might not be updated

Mounting (IV)

- **unmounting steps are thus**
  - data buffers flushing
    - OS writes cached data from memory to drives
  - closing of open file handles
    - OS closes any active file handles, ensuring no further file access
  - **updating file system metadata**
    - OS update metadata and journal on the file system (e.g. changes on the directory structure, file permissions, ownership)
  - **file system detaching**
    - OS detaches file system from the directory hierarchy, ensuring no more r/w possible operations on the drive

Command line tools for devices interactions

- **informations on connected (even not mounted) devices**
  - e.g. *dmesg, lsblk, fdisk, eventvwr, Get-WinEvent, device manager, Get-Disk, Get-Partition, wmic diskdrive, DiskPart, ...*
- **mounting in an integrity preserving way**
  - *mount -o ro /dev/sdb1 /mnt/usb*
- **mounting verification**
  - *mount | grep sdb1*
    - */dev/sdb1 on /mnt/usb type vfat (ro, noexec, nosuid, nodev)*
      - *ro:* read-only
      - *noexec*: prevents execution of binaries on the mounted file system
      - *nosuid:* disable the set-user ID bits on files on the mounted systems (cannot run with elevated privileges)
      - *nodev:* No special "device" files on the mounted system

Possible device types (I)

- **block device**
  - handle data in fixed size blocks
  - allow for random access
    - *find /dev -type b*
- **character device**
  - handle data as continuous stream
    - e.g. terminals and consoles
    - *ls –l /dev |grep '^c'*

Possible device types (II)

- **network device**
  - handle packet-based data transfer
  - use sockets and syscall to communicate
    - e.g. */dev/wlan0*
- **pseudo-device**
  - **interface to software feature**
  - **not directly related to actual HW**
    - but can emulate HW features
    - e.g. */dev/null, /dev/zero, /dev/random, /dev/loop*
- **special-purpose device**
  - very specific purpose for system function (managing system memory, interacting with the kernel, …)
  - e.g. */dev/mem /dev/kmem, /dev/console*

access to data at rest (Linux)

- **through the filesystem (after mounting process)**
  - access to device data from the mount point
  - adopt usual file system metaphors
    - files, directories, metadata, …
      - OS manage and update metadata
    - usual file operation (read, write, delete)
    - access control through file system rules (e.g. rwx) and usual operation (e.g. *ls, cp, mv, chmod*)
- **through direct access**
  - to the block device (e.g. */dev/sda1)*
  - allow access to raw (unstructured) blocks of data
    - risk of data corruption (no file system validation, access thourgh byte level commands like *dd* or *hexdump*)
    - foundamental for bit-by-bit copy

Linux distro interaction with devices

- **mount block devices as read-only**
  - many times by default
  - through command-line tool
    - *mount –o ro /dev/sdb1 /mnt/evidence1*
- **set the block device to read-only**
  - e.g. through command line tool like *blockdev* ioctls
    - *blockdev –setro /dev/sdb1*
- **or through GUI**
  - e.g. CAINE (11.x) provides *UnBlock* graphic tool
    - list of devices and write permission status
    - allow to change from/to Read-Only and Read-Write

disk image mounting

- **disk image = file containing bit-per-bit copy of an HD**
  - can be obtained through command-line tools like *dd*
- **manipulated as a normal physical device**
- **there are both GUIs and command line based tools**
  - *e.g. disk Image Mounter (CAINE 11.x)*
    - click and choose the file image
  - *imount* command line utility to mount volumes in *Encase* and *dd* format
    - *imount –v image.dd*

Write-protected set-up

- **attach the Device**
  - physically connect the storage device to your forensic workstation
  - preferably use a hardware write blocker to ensure no writes occur physically.
- **set block device to Read-Only**
- **mount the filesystem in Read-Only Mode**
- **check the mount status**
  - e.g. *mount | grep /mnt/<yourforensicdir>*
- **verify Read-Only Status of the Block Device**
  - **e.g.** `blockdev --getro /dev/sdX`
- **Note: device level protection override filesystem requests**
  - ...but may cause errors and inconsistences (journal/metadata)